

Föreläsning 2

DD1385

- Interface
- UML
- Lyssnarinterface

Förkunskap för denna föreläsning

- class
- Objekt
- Instansmetod, returtyp, parametrar och void
- Konstruktör, defaultkonstruktör
- Variabler och datatyper
- Arv
- Metod overriding / överskuggning
- Polymorfism?

Ett betalningssystem

Vi arbetar med ett e-handelssystem. Där kunden ska kunna:

- Välja produkter
- Gå till kassan
- Välja betalningsmetod
- Betala
- Få bekräftelse på köpet

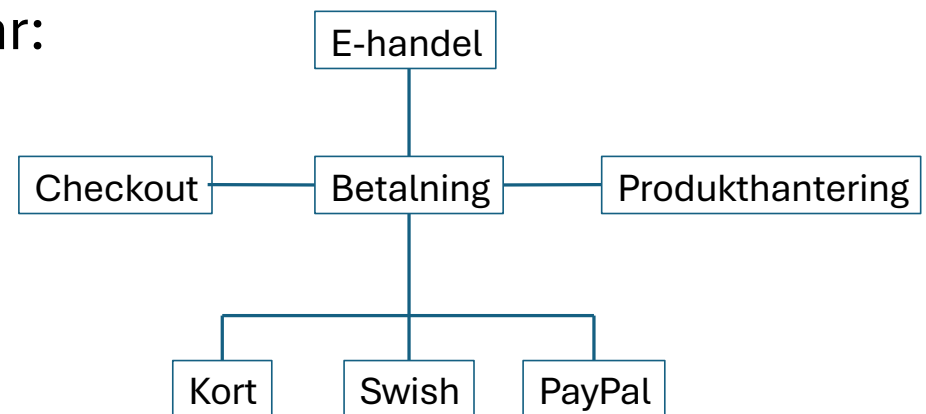
Ett program kan byggas av flera personer

I ett större projekt behöver inte en person skriva hela programmet. Därför kommunikation mellan inblandade programmerare är ytterst viktigt.

- Vi kan dela upp systemet i olika delar:

Exempel på ansvar:

- Programmerare A → produktkatalog
- Programmerare B → checkout
- Programmerare C → betalningar



Vi behöver kunna arbeta mot en överenskommelse

- Tänk att programmeraren som ansvarar för **checkout** säger:
"Jag behöver kunna be betalningsdelen att genomföra en betalning."
- Men personen som ansvarar för betalningsdelen kanske arbetar med:
 - kortbetalning
 - Swish
 - PayPal
 - Framtida betalningsmetoder
- Checkout-programmeraren ska **inte behöva vänta på att betalningsdelen är färdig**.
- De behöver bara komma överens om:
 - **"Om jag ger dig ett belopp ska du kunna genomföra en betalning."**
- Detta är ett **kontrakt mellan två delar av programmet**.

Vem bestämmer kontraktet?

- **Systemarkitekten/den ansvarige utvecklaren säger:**

"Alla betalningslösningar måste erbjuda följande funktion: pay()."

- Betalningsutvecklarna bestämmer sedan **hur** den funktionen fungerar.

Vad behöver checkout veta?

När kunden trycker på **Betala** behöver checkout egentligen veta ganska lite.

- **Viktig information:**

- Order: 12345
- Belopp: 499 SEK

- Checkout behöver veta:

- **Vad ska betalas?**
- **Hur mycket ska betalas?**

- Checkout behöver inte veta:

- Hur en kortbetalning fungerar
- Hur Swish fungerar
- Hur PayPal fungerar
- Vilka API-anrop som krävs

Kärnan: Checkout bryr sig om **vad** som ska göras, inte **hur** det görs.

Men betalningsmetoderna fungerar olika

Kort:

- Kortnummer
- Verifiering
- 3-D Secure
- Kortleverantörens API

Swish

- Telefonnummer
- Betalningsförfrågan
- Bekräftelse
- Swish API

PayPal

- Konto
- Session/token
- Redirect
- PayPal API

Men alla metoder har samma mål: Genomför en betalning på ett visst belopp.

Scenario: kortbetalning

- **Kunden köper en produkt för 499 SEK.**
- Checkout: "Betala 499 SEK"
- CreditCardPayment
 - Kortspezifisk logik
 - Kortleverantörens API
 - Betalning genomförd
- **Viktig poäng:**
- Checkout behöver inte känna till stegen mellan CreditCardPayment och kortleverantören.

Scenario: Swish

- **Samma order men swish som betalningsmetod: 499 SEK**
- Checkout: "Betala 499 SEK"
- SwishPayment
 - Swish-specifik logik
 - Swish API
 - Betalning genomförd
- Här behöver Checkout inte heller veta detaljer i SwishPayment
- **Det som skiljer mellan olika metoder är implementationen.**

Problemet

- Vi vill kunna skriva:
`paymentMethod.pay(499);`
- Men vi vill **inte** att checkout ska behöva göra:

```
if (paymentType == SWISH) {  
    // Swish-logik  
} else if (paymentType == CREDIT_CARD) {  
    // kortlogik  
} else if (paymentType == PAYPAL) {  
    // PayPal-logik  
}
```

Varför?

För då måste checkout känna till alla betalningsmetoder. Och varje gång vi lägger till en ny betalningsmetod måste checkout ändras.

Lösning: Interface

- **Lösningen: Interface**
- Vi definierar ett **kontrakt**:

```
interface PaymentMethod {  
    void pay(double amount);  
}
```

- Det betyder:
 - **Alla PaymentMethod måste kunna genomföra pay(). (Tvingande!!!)**
 - Interfacet lägger sig inte i **Hur** betalningen ska genomföras. Det lämnar vi till implementationen av metoden `pay`.

Olika implementationer

```
class CreditCardPayment implements PaymentMethod {  
    public void pay(double amount) {  
        // kortspezifisk logik  
    }  
}
```

```
class SwishPayment implements PaymentMethod {  
    public void pay(double amount) {  
        // Swish-specifik logik  
    }  
}
```

Båda **måste** implementera `pay(double amount)` men de **får** göra det på **helt olika sätt**.

Vad har vi uppnått?

- Nu kan checkout arbeta mot **interface**:

```
void processPayment(PaymentMethod method, double b) {  
    method.pay(b);  
}
```

- Vi kan göra anroperna:

- `processPayment(new CreditCardPayment(), 499);`
- `processPayment(new SwishPayment(), 499);`

- Checkout behöver inte veta vilken konkret klass den fått.

- **Checkout vet bara:**

"Det här objektet är en PaymentMethod och kan därför pay()."

Viktig abstraktionspoäng

Information kan vara viktig för implementationen utan att vara relevant för den som använder implementationen.

Exempel:

- **Checkout bryr sig endast om:** 499 SEK
- **CreditCardPayment kan behöva:** kortinformation, 3-D Secure, API credentials,...
- **SwishPayment kan behöva:** telefonnummer, Swish-session, API-information, ...
- Den informationen är **inte oviktig** men är bara **inte relevant på checkout-nivån**.

Den stora poängen

- **Interface = kontrakt**

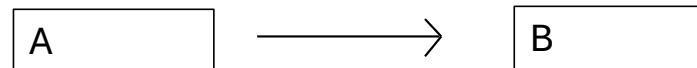
```
interface PaymentMethod {  
    void pay(double amount);  
}
```

- **Checkout säger:** "Jag behöver något som kan betala."
- **Interfacet säger:** "Alla som är PaymentMethod måste kunna pay()."
- **CreditCardPayment säger:** "Jag skriver under kontraktet och bestämmer själv hur kortbetalning fungerar."
- **SwishPayment säger:** "Jag skriver under kontraktet och bestämmer själv hur Swish-betalning fungerar."
- **Samma kontrakt — olika implementationer.**

UML

Unified Modelling Language: Beskriva relation mellan olika klasser med UML.

- Komposition: En enkel heldragen linje med spetsig pil. Klass A har en referens till B



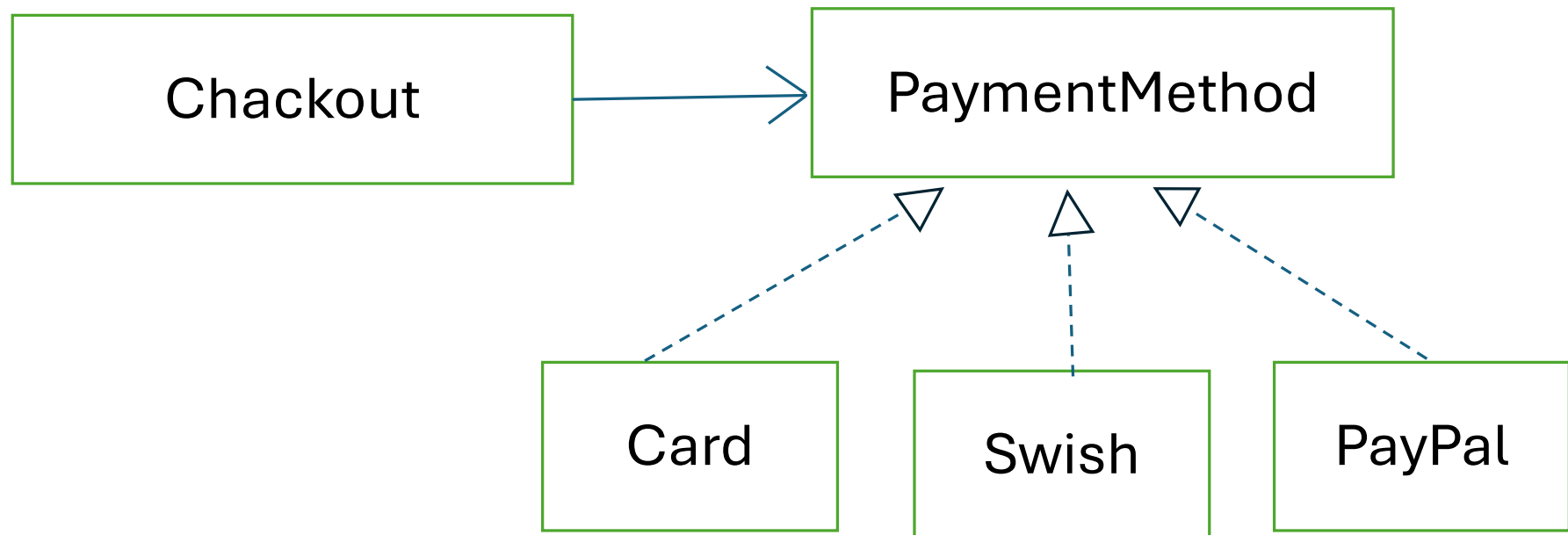
- Arv: Heldragen pil med triangelform spets används för att beskriva arv: subclass ärver från superklass



- Implementera: Streckad pil med triangelform spets används för att beskriva relation mellan implementerande klass och interface



UML



Lyssnarinterface

Vi tittar på några exempel AWTDemo